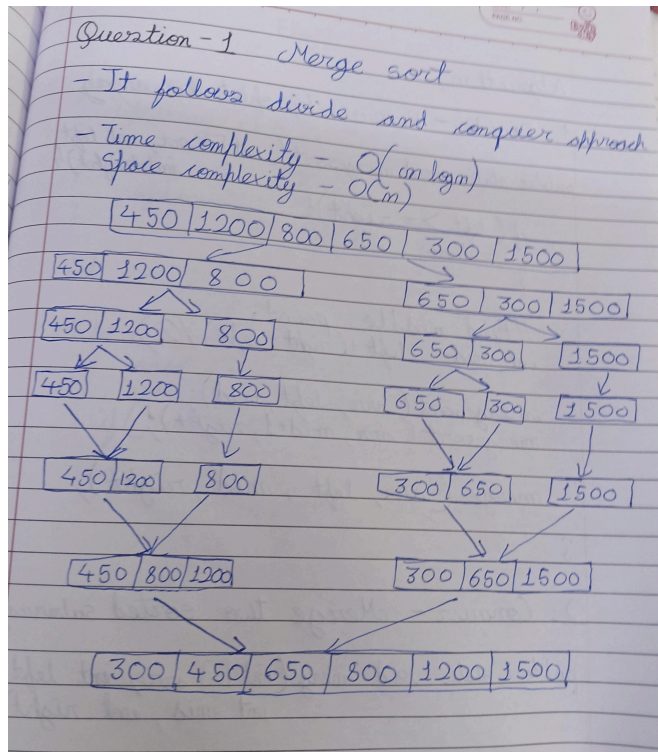


# Question 1 A Merge sort



Algorithm

- Divide - Recursively split the array

```

public static void mergesort (int[] arr, int left,
                             int right) {
    if (left >= right) {
        return;
    }

    // Find middle point
    int mid = left + (right - left) / 2;

    mergesort (arr, left, mid);
    mergesort (arr, mid + 1, right);
    merge (arr, left, mid, right);
}
    
```

- Conquer - Merge two sorted subarrays

```

public static void merge (int[] arr, int left,
                          int mid, int right) {
    int m1 = mid - left + 1;
    int m2 = right - mid;
    int[] leftarray = new int[m1];
    int[] rightarray = new int[m2];
    
```

```

for (int i=0; i<m1; i++) {
    leftarray[i] = arr[left+i];
}
for (int j=0; j<m2; j++) {
    rightarray[j] = arr[mid+1+j];
}

int i=0, j=0, k=left;
while (i<m1 || j<m2) {
    if (leftarray[i] <= rightarray[j]) {
        arr[k] = leftarray[i];
        i++;
    } else {
        arr[k] = rightarray[j];
        j++;
    }
    k++;
}

while (i<m1) {
    arr[k] = leftarray[i];
    i++; k++;
}
while (j<m2) {
    arr[k] = rightarray[j];
    j++; k++;
}
}

```

```

import java.util.Arrays;
import java.util.Scanner;
public class Main{

    public static void mergeSort(int[] arr) {
        if (arr == null || arr.length <= 1) {
            return;
        }
        sort(arr, 0, arr.length - 1);
    }

    private static void sort(int[] arr, int left, int right) {
        if (left < right) {
            int mid = left + (right - left) / 2;
            sort(arr, left, mid);
            sort(arr, mid + 1, right);
            merge(arr, left, mid, right);
        }
    }
}

```

```

private static void merge(int[] arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int[] leftArray = new int[n1];
    int[] rightArray = new int[n2];

    for (int i = 0; i < n1; ++i) {
        leftArray[i] = arr[left + i];
    }
    for (int j = 0; j < n2; ++j) {
        rightArray[j] = arr[mid + 1 + j];
    }

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (leftArray[i] <= rightArray[j]) {
            arr[k] = leftArray[i];
            i++;
        } else {
            arr[k] = rightArray[j];
            j++;
        }
        k++;
    }
    while (i < n1) {
        arr[k] = leftArray[i];
        i++;
        k++;
    }
    while (j < n2) {
        arr[k] = rightArray[j];
        j++;
        k++;
    }
}

```

```

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    System.out.print("Enter the number of packages: ");
    int n = scanner.nextInt();

    int[] arr = new int[n];

    System.out.println("\nEnter " + n + " integers:\n");
    for (int i = 0; i < n; i++) {
        System.out.print("Element " + (i + 1) + ": ");
        arr[i] = scanner.nextInt();
    }
    scanner.close();
    mergeSort(arr);
    System.out.println("Sorted Array: " + Arrays.toString(arr));
}

```

```
Enter the number of packages: 6

Enter 6 integers:

Element 1: 450
Element 2: 1200
Element 3: 800
Element 4: 650
Element 5: 300
Element 6: 1500
Sorted Array:  [300, 450, 650, 800, 1200, 1500]
```

## Question 1 B

```
import java.util.Arrays;
import java.util.Scanner;
public class Main {
    public static void mergeSort(int[] arr, int left, int right) {
        if (left < right) {
            int mid = left + (right - left) / 2;
            mergeSort(arr, left, mid);
            mergeSort(arr, mid + 1, right);
            merge(arr, left, mid, right);
        }
    }
    private static void merge(int[] arr, int left, int mid, int right) {
        int n1 = mid - left + 1, n2 = right - mid;
        int[] L = new int[n1];
        int[] R = new int[n2];
        System.arraycopy(arr, left, L, 0, n1);
        System.arraycopy(arr, mid + 1, R, 0, n2);
        int i = 0, j = 0, k = left;
        while (i < n1 && j < n2) {
            if (L[i] <= R[j]) {
                arr[k] = L[i];
                i++;
            } else {
                arr[k] = R[j];
                j++;
            }
            k++;
        }
        while (i < n1) {
            arr[k] = L[i];
            i++; k++;
        }
        while (j < n2) {
            arr[k] = R[j];
            j++; k++;
        }
    }
}
```

```

public static void main(String[] args){
    Scanner scanner = new Scanner(System.in);
    System.out.print("\nEnter the number of processing times : ");
    int n = scanner.nextInt();
    int[] times = new int[n];
    System.out.println("\nEnter " + n + " integers:\n");
    for (int i = 0; i < n; i++) {
        System.out.print("Element " + (i + 1) + ": ");
        times[i] = scanner.nextInt();
    }
    scanner.close();
    mergeSort(times, 0, n - 1);
    System.out.println("\nSorted Array: " + Arrays.toString(times));

    // 2. Find Median
    int median, medianIndex;
    if (n % 2 != 0) {
        median = times[n / 2];
        medianIndex = n / 2;
    } else {
        median = (times[(n / 2) - 1] + times[n / 2]) / 2;
        medianIndex = (n / 2) - 1;
    }
    System.out.println("\nMedian: " + median);
}

```

```

    // 3. Count orders strictly greater than the median
    int countAboveMedian = 0;
    for (int i = n - 1; i >= 0; i--) {
        if (times[i] > median) {
            countAboveMedian++;
        } else {
            break;
        }
    }
    System.out.println("\nOrders Above Median: " + countAboveMedian);

    // 4. Calculate Difference between max and min processing time
    int minTime = times[0], maxTime = times[n - 1];
    int difference = maxTime - minTime;
    System.out.println("\nDifference: " + difference);
}
}

```

Enter the number of processing times : 7

Enter 7 integers:

Element 1: 12

Element 2: 25

Element 3: 18

Element 4: 30

Element 5: 15

Element 6: 22

Element 7: 40

Sorted Array: [12, 15, 18, 22, 25, 30, 40]

Median: 22

Orders Above Median: 3

Difference: 28



## Question 2 A Quick sort

Question - 2 Quick Sort

- It follows divide and Conquer via Partitioning.

Time complexity -  $O(N \log N)$   
Space complexity -  $O(\log N)$

arr = 

|     |    |    |     |    |    |
|-----|----|----|-----|----|----|
| 120 | 45 | 78 | 200 | 60 | 95 |
|-----|----|----|-----|----|----|

  
i↑  
pivot = 95

Compare 120:  $120 > 95$  do nothing  
Compare 45:  $45 < 95 \rightarrow$  Increment i swap 120 to 45  
Compare 78:  $78 < 95 \rightarrow$  Increment i swap 120 to 78  
Compare 200:  $200 > 95$  do nothing  
Compare 60:  $60 < 95 \rightarrow$  Increment i swap 120 and 60

Final array become

|    |    |  |  |  |  |
|----|----|--|--|--|--|
| 45 | 78 |  |  |  |  |
|----|----|--|--|--|--|

|    |    |    |    |     |     |
|----|----|----|----|-----|-----|
| 45 | 60 | 78 | 95 | 120 | 200 |
|----|----|----|----|-----|-----|

Algorithm

```
public static void quicksort(int[] arr,
                             int low, int high){
    if (low < high){
        int pi = partition(arr, low, high);
        quicksort(arr, low, pi-1);
        quicksort(arr, pi+1, high);
    }
}

public static int partition(int[] arr, int low, int high){
    int pivot = arr[high];
    int i = low-1;

    for (int j = low; j < high; j++){
        if (arr[j] <= pivot){
            i++;
            swap arr[i] and
        }
    }
    swap pivot and arr[i+1]
    return i+1;
}
```

```

import java.util.Arrays;
import java.util.Scanner;

public class Main {

    public static void quickSort(int[] arr, int low, int high) {
        if (low < high) {
            int partitionIndex = partition(arr, low, high);
            quickSort(arr, low, partitionIndex - 1);
            quickSort(arr, partitionIndex + 1, high);
        }
    }
}

```

```

private static int partition(int[] arr, int low, int high) {
    int pivot = arr[high] , i = (low - 1);
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    return i + 1;
}

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    System.out.print("Enter the number of elements you want to sort: ");
    int n = scanner.nextInt();
    int[] arr = new int[n];
    System.out.println("Enter " + n + " integers:");
    for (int i = 0; i < n; i++) {
        System.out.print("Element " + (i + 1) + ": ");
        arr[i] = scanner.nextInt();
    }
    scanner.close();
    quickSort(arr, 0, arr.length - 1);
    System.out.println("Sorted Array: " + Arrays.toString(arr));
}
}

```



```
Enter the number of response time: 6

Enter 6 integers:

Element 1: 120
Element 2: 45
Element 3: 78
Element 4: 200
Element 5: 60
Element 6: 95

Sorted Array:    [45, 60, 78, 95, 120, 200]
```

## Question 2 B

```
import java.util.Arrays;
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter the number of trade values: ");
        int n = scanner.nextInt();
        int[] trades = new int[n];
        System.out.println("\nEnter " + n + " integers:\n");
        for (int i = 0; i < n; i++) {
            System.out.print("Element " + (i + 1) + ": ");
            trades[i] = scanner.nextInt();
        }
        scanner.close();
        quickSort(trades, 0, n-1);
        System.out.println("\nSorted Array:    " + Arrays.toString(trades));

        System.out.print("Top 5:");
        long Sum = 0;
        for (int i = 0; i < 5; i++) {
            System.out.print(" " + trades[i]);
            Sum += trades[i];
        }

        System.out.println();
        double Averageof5 = (double) Sum / 5;
        System.out.printf("Average of Top 5: %.2f\n", Averageof5);
        int sumAll = Arrays.stream(trades).sum();
        float avgAll = (float) sumAll/n;
        int Count = 0;
        for (int i = 0; i < n; i++) {
            if (trades[i] > avgAll) {
                Count++;
            }
        }
        System.out.println("Values Above Overall Average: " + Count);
    }
}
```

```

private static void quickSort(int[] arr, int low, int high) {
    if (low < high) {
        int Index = partition(arr, low, high);
        quickSort(arr, low, Index - 1);
        quickSort(arr, Index + 1, high);
    }
}

private static int partition(int[] arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] > pivot) {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    return i + 1;
}
}

```

Enter the number of trade values: 8

Enter 8 integers:

Element 1: 500

Element 2: 1200

Element 3: 700

Element 4: 2000

Element 5: 900

Element 6: 1500

Element 7: 3000

Element 8: 2500

Sorted Array: [3000, 2500, 2000, 1500, 1200, 900, 700, 500]

Top 5: 3000 2500 2000 1500 1200

Average of Top 5: 2040.00

Values Above Overall Average: 3

# Question 3A Heap sort

Q-3 Heap Sort

Time Complexity =  $O(N \log N)$   
Space Complexity =  $O(1)$

```
public static void heapsort ( int [] arr ) {  
    int m = arr.length();  
    for ( int i = m/2 - 1; i >= 0; i-- ) {  
        maxHeapify ( arr, m, i );  
    }  
    for ( int i = m-1; i > 0; i-- ) {  
        int temp = arr[0];  
        arr[0] = arr[i];  
        arr[i] = temp;  
        maxHeapify ( arr, i, 0 );  
    }  
}
```

```
public static void maxHeapify ( int [] arr, int m,  
                                int i ) {  
    int largest = i;  
    int left = 2 * i + 1;  
    int right = 2 * i + 2;  
    if ( left < m && arr[left] > arr[largest] ) {  
        largest = left;  
    }  
    if ( right < m && arr[right] > arr[largest] ) {  
        largest = right;  
    }  
    if ( largest != i ) {  
        int swap = arr[i];  
        arr[i] = arr[largest];  
        arr[largest] = swap;  
        maxHeapify ( arr, m, largest );  
    }  
}
```

```

import java.util.Arrays;
import java.util.Scanner;

public class Main {
    public static void heapSort(int[] arr) {
        int n = arr.length;
        for (int i = n / 2 - 1; i >= 0; i--) {
            heapify(arr, n, i);
        }
        for (int i = n - 1; i > 0; i--) {
            int temp = arr[0];
            arr[0] = arr[i];
            arr[i] = temp;
            heapify(arr, i, 0);
        }
    }

    private static void heapify(int[] arr, int heapSize, int i) {
        int largest = i, left = 2 * i + 1, right = 2 * i + 2;
        if (left < heapSize && arr[left] > arr[largest]) {
            largest = left;
        }
        if (right < heapSize && arr[right] > arr[largest]) {
            largest = right;
        }
        if (largest != i) {
            int swap = arr[i];
            arr[i] = arr[largest];
            arr[largest] = swap;
            heapify(arr, heapSize, largest);
        }
    }
}

```

```

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter the number of scores: ");
        int n = scanner.nextInt();
        int[] arr = new int[n];
        System.out.println("\nEnter " + n + " integers:\n");
        for (int i = 0; i < n; i++) {
            System.out.print("Element " + (i + 1) + ": ");
            arr[i] = scanner.nextInt();
        }
        scanner.close();
        heapSort(arr);
        System.out.println("\nSorted Array: " + Arrays.toString(arr));
    }
}

```

```
Enter the number of scores: 6

Enter 6 integers:

Element 1: 850
Element 2: 1200
Element 3: 950
Element 4: 600
Element 5: 1500
Element 6: 1000

Sorted Array:  [600, 850, 950, 1000, 1200, 1500]
```

## Question 3 B

```
public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    System.out.print("Enter the number of response time: ");
    int n = scanner.nextInt();
    int[] arr = new int[n];
    System.out.println("\nEnter " + n + " integers:\n");
    for (int i = 0; i < n; i++) {
        System.out.print("Element " + (i + 1) + ": ");
        arr[i] = scanner.nextInt();
    }
    scanner.close();
    heapSort(arr);
    System.out.println("\nSorted Array:  " + Arrays.toString(arr));
    System.out.println("Fastest : \t" + arr[0]);
    System.out.println("Slowest : \t" + arr[n-1]);
    //3. Average
    int sum = Arrays.stream(arr).sum();
    float avg = (float) sum / n;
    System.out.println("Average : \t " + avg);

    int count = 0;
    for (int i = 0; i < n; i++) {
        if (arr[i] <= avg)
            count++;
    }
    System.out.println("Cases faster than average : \t " + count);

    // Calculate percentage using float division
    float percentage = ((float) count * 100) / n;
    System.out.println("percentage : \t " + percentage + "%");
}
}
```

```

import java.util.Arrays;
import java.util.Scanner;

public class Main {

    public static void heapSort(int[] arr) {
        int n = arr.length;
        for (int i = n / 2 - 1; i >= 0; i--) {
            heapify(arr, n, i);
        }
        for (int i = n - 1; i > 0; i--) {
            int temp = arr[0];
            arr[0] = arr[i];
            arr[i] = temp;
            heapify(arr, i, 0);
        }
    }

    private static void heapify(int[] arr, int heapSize, int i) {
        int largest = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;

        if (left < heapSize && arr[left] > arr[largest])
            largest = left;

        if (right < heapSize && arr[right] > arr[largest])
            largest = right;

        if (largest != i) {
            int swap = arr[i];
            arr[i] = arr[largest];
            arr[largest] = swap;
            heapify(arr, heapSize, largest);
        }
    }
}

```

Enter the number of response time: 8

Enter 8 integers:

Element 1: 12

Element 2: 7

Element 3: 15

Element 4: 9

Element 5: 20

Element 6: 5

Element 7: 10

Element 8: 8

Sorted Array: [5, 7, 8, 9, 10, 12, 15, 20]

Fastest : 5

Slowest : 20

Average : 10.75

Cases faster than average : 5

percentage : 62.5%